

UConn · School of Business

Lab 3 — Build a Live Data Robot

Two tiny APIs · one robot · all in VS Code

OPIM 5512 · Module 3 · Operational ML

Tonight your code stops needing you.

The deal

- New England's grid publishes electricity demand **every 5 minutes**. Your campus airport reports the weather **every 5 minutes** too.
- Partner A writes a tiny **energy API** (ISO-NE). Partner B writes a tiny **weather API** (National Weather Service).
- Same loop as Labs 1–2: branch → commit → push → pull request → review → merge.
- Then a **robot** (GitHub Actions) calls both APIs every 5 minutes and saves every new row — **forever, free, while you sleep**.

First ~15 minutes: why APIs and robots. Then you build. Everything tonight happens in VS Code.

An API is a promise: hide the mess

WHAT ISO-NE ACTUALLY SENDS

```
"C","Real-Time Five-Minute System Load"
"C","Report generated: 09/24/2026 10:59"
"H","Date/Time","Total Load","Native..."
"H","Date/Time","MW","MW","MW"
"D","09/24/2026 10:50:00",8751.362,...
"D","09/24/2026 10:55:00",8702.394,...
"T","132 lines"
# ...and only if you visited the report page
# first for a cookie + sent it as the Referer
```



WHAT YOUR API RETURNS

```
load = get_five_minute_load()
load.tail(2)
```

timestamp	total_load_mw	...
2026-09-24 10:50:00	8751.36	...
2026-09-24 10:55:00	8702.39	...

An API is a clean promise about what you get back. Nobody downstream should ever have to know about cookies, referers, or row codes.

Two feeds, two messes, two APIs

PARTNER A — ENERGY · SRC/ISONE.PY

```
session = requests.Session()
# TODO 1: visit the page -> get a cookie
session.get(REPORT_PAGE, timeout=30)
# TODO 2: ask for the CSV, say where you came from
resp = session.get(url, headers={"Referer": REPORT_PAGE})
# TODO 3: keep only the '"D"' data rows
for line in resp.text.splitlines():
    if line.startswith('"D"'):
        data_rows.append(line)
```

PARTNER B — WEATHER · SRC/NWS.PY

```
# TODO 1: identify yourself, get the JSON
resp = requests.get(url, headers={
    "User-Agent": USER_AGENT}, timeout=30)

# TODO 2: metric -> the units we use
"temp_f": c_to_f(p["temperature"]["value"]),
"wind_mph": kmh_to_mph(p["windSpeed"]["value"]),
# ...dewpoint_f, humidity_pct
```

3 small TODOs for A, 2 for B. Then delete the raise NotImplementedError line and run it: real rows from right now.

The robot: GitHub Actions on a schedule

.GITHUB/WORKFLOWS/COLLECT.YML (GIVEN)

```
on:
  schedule:
    - cron: "*/5 * * * *" # every 5 min
  workflow_dispatch:      # + a Run button

steps:
  - # check out the code (main)
  - run: python src/collect.py
  - # commit new rows to the data branch
```

WHERE THINGS LIVE

- **main** = the code. Protected. Changes arrive only through a reviewed PR.
- **data** = the data. The robot creates it on its first run and is the only one who commits there.
- That's why 288 robot commits a day never clutter **your** network graph.

The cloud runs your code now — not you. Public repos get unlimited free robot minutes. Keep your repo PUBLIC.

Why a late robot is still right

GitHub's "every 5 minutes" is **roughly** every 5 minutes — runs start late, and sometimes get skipped when it's busy. So the robot never asks "*what happened in the last 5 minutes?*" It pulls **the whole recent window** every time and **upserts**:

robot run	fetches	new rows kept	what happened
11:05 (on time)	422	+1	one fresh 5-minute row
11:10 — skipped	—	—	GitHub was busy
11:17 (late)	424	+2	caught up both missing rows
11:18 (ran twice)	424	+0	"No new rows this run." — correct

New rows get added. Rows it has seen get refreshed. Nothing is ever lost or doubled. That's how real data pipelines survive an unreliable clock.

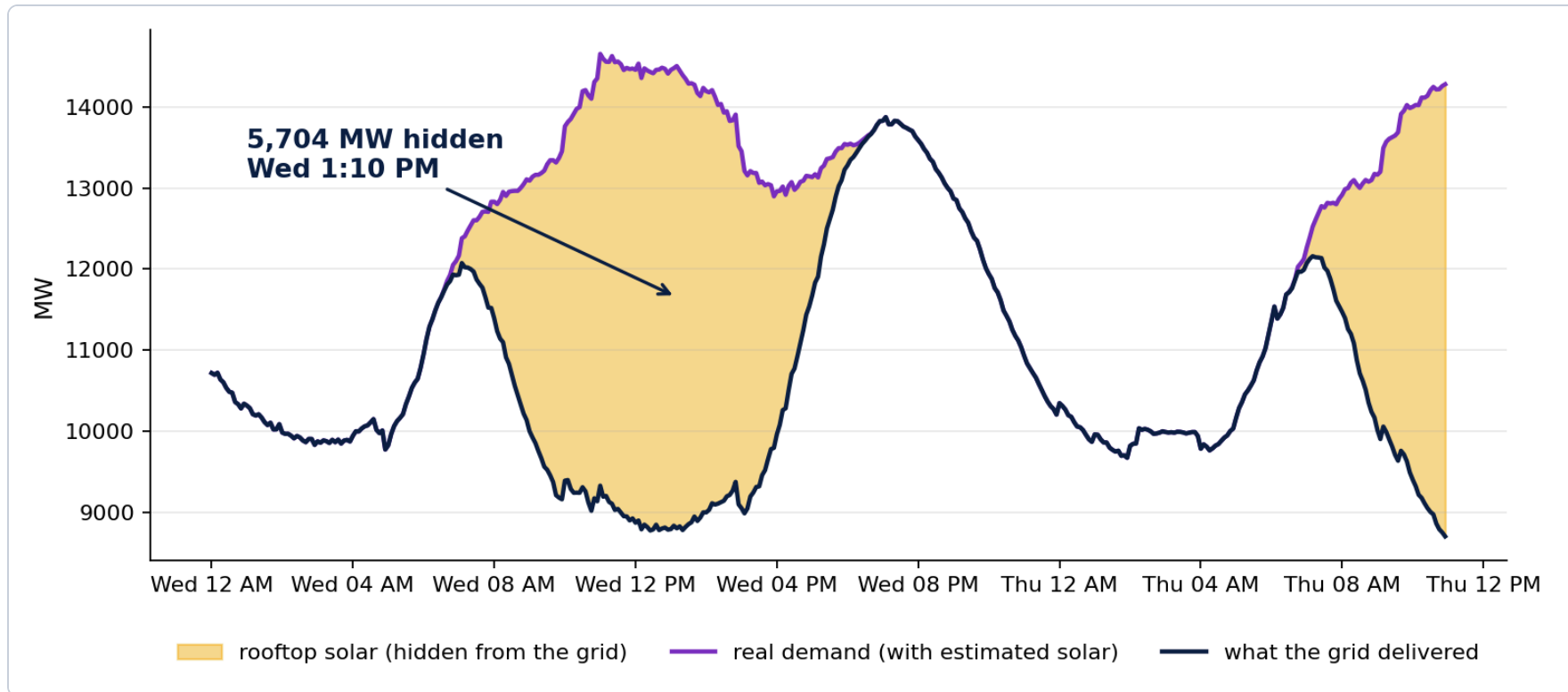
How live is “live”? We measured it.

Polled every minute for 30 minutes on Sept 24. The question isn't just *how often* — it's *how late* each reading arrives:

feed	a reading every...	it shows up...	so the robot sees new data...
ISO-NE system load	5 minutes	1–3 minutes late	almost every run
Weather KBDL / KBDR	5 minutes (+ the classic :51 hourly report)	in batches, 8–24 minutes late	most runs, a few at a time

Published ≠ arrived. Live data always runs a little behind the clock. Your notebook measures exactly how far.

The payoff: demand the grid never sees



Rooftop solar hides thousands of megawatts at midday. ISO-NE reports both loads every 5 minutes. Your notebook plots this gap from YOUR robot's data.

Stack check & roles

- **GitHub Desktop** signed in · **VS Code** with the **Python** + **Jupyter** extensions
- **Pick roles now** — Partner A = **energy** (src/isone.py), Partner B = **weather** (src/nws.py)
- **Stamford:** Partner B changes the station to **KBDR** (Sikorsky). The grid is shared; the weather is local.
- **Laptop Python fighting you?** Don't lose the lab to setup: **Code** → **Codespaces** → **Create** = VS Code in the browser, everything installed.

Odd one out pairs with Dave. Online / solo? Be both A and B with two branches, approvals = 0.

ACT 1 • SET UP (STEPS 1–6)

Make the shared repo — from the template

- 1 • Pair up — Partner A (energy) / Partner B (weather)
- 2 • Partner A: template repo → Use this template → new **PUBLIC** repo, owner = you
- 3 • Partner A: Settings → Collaborators → add B → **B ACCEPTS the invite now**
- 4 • Partner A: Settings → Rules → ruleset on main: require a PR + 1 approval
- 5 • Both: clone the repo ONCE in GitHub Desktop
- 6 • Each: New branch (dev-energy / dev-weather) → Publish → Fetch — **before touching any file**

The template already has the robot, the tests, and the notebook. You write the two APIs.

Fill the TODOs, run it, ship it

- 7 • GitHub Desktop → Repository → Open in Visual Studio Code (check: your dev- branch)
- 8 • Fill the TODOs in **your** file → delete the `raise NotImplementedError` line
- 9 • Terminal: `pip install -r requirements.txt` → `python src/isone.py` / `python src/nws.py` → **live rows**
- 10 • GitHub Desktop: commit on your dev- branch → Push
- 11 • Compare & pull request → Reviewers → your partner → **watch the check run**
- 12 • Open your PARTNER's PR → read their function + look for the green ✓ → Approve

Green ✓ = your API passed. Your partner's tests say SKIPPED until they merge. Red X? Click Details — the FAILED line says what's wrong in plain English.

Hand your code to the cloud

- 13** • Merge both PRs → Delete branch
- 14** • **Actions** tab → **collect** → **Run workflow** (it also runs itself ~every 5 minutes — this skips the wait)
- 15** • When it's ✓: Code → switch the branch menu to **data** → open both CSVs — **the robot's work**
- 16** • Both: GitHub Desktop → main → Fetch → Pull

Every commit on the data branch is by lab3-bot. Your main branch stayed clean — code and data live apart.

Your data, growing

- 17 • VS Code → notebooks/Lab3_Watch_Your_Live_Data.ipynb → Select Kernel → set OWNER + REPO → Run All
- 18 • The solar-gap plot: when is rooftop solar hiding the most demand?
- 19 • Wait an hour, Run All again — more rows. Tomorrow — a lot more.
- 20 • Submit: screenshot **Insights** → **Network** + the **data** branch's commit list

You didn't just analyze data tonight. You built the thing that collects it.

Good advice

- **NotImplementedError?** You filled the TODOs but didn't delete the `raise` line.
- **Red X "temp_f=16.0 but the API says 15 C = 59.0 F"?** You stored Celsius — wrap it in `c_to_f(...)`.
- **Committed on main by accident?** New branch → "Bring my changes to the new branch" → yes.
- **"No new rows this run"** is correct, not broken — the feed hadn't published anything newer.
- **python not found (Windows)?** Use `py`. Still stuck after 5 minutes? Switch to **Codespaces**.

Definition of done

- ☐ Repo is **public**; both partners are collaborators; branch protection on main
- ☐ **2 merged pull requests** (one each), both with a green ✓, branches deleted
- ☐ The **data** branch exists with **both** isone_5min.csv and weather_obs.csv
- ☐ A **network graph** showing the loop going both ways
- ☐ (*optional bonus*) the notebook's solar-gap plot + one sentence on what it shows

Leave the robot running. Next module, you'll model the data it collects while you were away.